

Escuela Politécnica Nacional

Nombre: Kevin Eduardo Garcia Rodriguez

Tema: [Tarea 11] Ejercicios Unidad 04-D | Gauss-Jacobi y Gauss-Seidel

Repositorio GIT: <https://github.com/Nattyrd/Metodos-Numericos-2025B>

Ejercicio 1

Encuentre las primeras dos iteraciones del método de Jacobi para los siguientes sistemas lineales, por medio de $\mathbf{x}(0) = 0$:

```
In [1]: import numpy as np

def resolver_sistema_lineal_3d(A, b):
    """
    Resuelve un sistema de ecuaciones lineales 3x3 de la forma Ax = b.

    Args:
        A (np.array): La matriz de coeficientes (3x3).
        b (np.array): El vector de constantes (1D, 3 elementos).

    Returns:
        np.array: El vector de solución (1D, 3 elementos) si existe.
        Devuelve un mensaje de error si la matriz es singular (sin sol
    """
    try:
        # np.linalg.solve() resuelve el sistema de forma eficiente.
        x = np.linalg.solve(A, b)
        return x
    except np.linalg.LinAlgError:
        # Si la matriz es singular (determinante cero), el sistema no tiene
        # una solución única. Podría tener infinitas soluciones o ninguna.
        return "El sistema no tiene una solución única (infinitas soluciones o n
```

$$3x_1 - x_2 + x_3 = 1$$

$$3x_1 + 6x_2 + 2x_3 = 0$$

$$3x_1 + 3x_2 + 7x_3 = 4$$

```
In [6]: # Definimos la matriz y el vector del problema
A = np.array([
    [3, -1, 1],
    [3, 6, 2],
    [3, 3, 7]
])

b = np.array([1, 0, 4])

# Resolvemos el sistema
solucion = resolver_sistema_lineal_3d(A, b)
```

```
# Imprimimos la solución y la interpretación geométrica
print("La matriz de coeficientes es:")
print(A)
print("\nEl vector de constantes es:")
print(b)

print("\n--- Solución del sistema ---")
if isinstance(solucion, np.ndarray):
    print("El sistema tiene una solución única:")
    print(f"x1 = {solucion[0]:.4f}")
    print(f"x2 = {solucion[1]:.4f}")
    print(f"x3 = {solucion[2]:.4f}")
    print("\nInterpretación geométrica: Los tres planos se intersectan en un único punto.")
else:
    print(solucion)
    print("\nInterpretación geométrica: infinitas soluciones o no tienen un punto común.")
```

La matriz de coeficientes es:

```
[[ 3 -1  1]
 [ 3  6  2]
 [ 3  3  7]]
```

El vector de constantes es:

```
[1 0 4]
```

--- Solución del sistema ---

El sistema tiene una solución única:

x1 = 0.0351

x2 = -0.2368

x3 = 0.6579

Interpretación geométrica: Los tres planos se intersectan en un único punto.

$$10x_1 - x_2 = 9$$

$$-x_1 + 10x_2 - 2x_3 = 7$$

$$-2x_2 + 10x_3 = 6$$

```
In [3]: # Definimos la matriz y el vector del problema
A = np.array([
    [10, -1, 0],
    [-1, 10, -2],
    [0, -2, 10]
])

b = np.array([9, 7, 6])

# Resolvemos el sistema
solucion = resolver_sistema_lineal_3d(A, b)

# Imprimimos la solución y la interpretación geométrica
print("La matriz de coeficientes es:")
print(A)
print("\nEl vector de constantes es:")
print(b)

print("\n--- Solución del sistema ---")
if isinstance(solucion, np.ndarray):
    print("El sistema tiene una solución única:")
```

```

    print(f"x1 = {solucion[0]:.4f}")
    print(f"x2 = {solucion[1]:.4f}")
    print(f"x3 = {solucion[2]:.4f}")
    print("\nInterpretación geométrica: Los tres planos se intersecan en un único
else:
    print(solucion)
    print("\nInterpretación geométrica: infinitas soluciones o no tienen un punto")

```

La matriz de coeficientes es:

```

[[10 -1  0]
 [-1 10 -2]
 [ 0 -2 10]]

```

El vector de constantes es:

```

[9 7 6]

```

--- Solución del sistema ---

El sistema tiene una solución única:

x1 = 0.9958

x2 = 0.9579

x3 = 0.7916

Interpretación geométrica: Los tres planos se intersecan en un único punto.

$$10x_1 + 5x_2 = 6$$

$$5x_1 + 10x_2 - 4x_3 = 25$$

$$-4x_2 + 8x_3 - x_4 = -11$$

$$-x_3 + 5x_4 = -11$$

```

In [4]: # Definimos la matriz y el vector del problema
A = np.array([
    [10, 5, 0, 0],
    [5, 10, -4, 0],
    [0, -4, 8, -1],
    [0, 0, -1, 5]
])

b = np.array([6, 25, -11, -11])

# Resolvemos el sistema
solucion = resolver_sistema_lineal_3d(A, b)

# Imprimimos la solución y la interpretación geométrica
print("La matriz de coeficientes es:")
print(A)
print("\nEl vector de constantes es:")
print(b)

print("\n--- Solución del sistema ---")
if isinstance(solucion, np.ndarray):
    print("El sistema tiene una solución única:")
    print(f"x1 = {solucion[0]:.4f}")
    print(f"x2 = {solucion[1]:.4f}")
    print(f"x3 = {solucion[2]:.4f}")
    print("\nInterpretación geométrica: Los tres planos se intersecan en un único
else:

```

```
print(solucion)
print("\nInterpretación geométrica: infinitas soluciones o no tienen un punt
```

La matriz de coeficientes es:

```
[[10  5  0  0]
 [ 5 10 -4  0]
 [ 0 -4  8 -1]
 [ 0  0 -1  5]]
```

El vector de constantes es:

```
[ 6  25 -11 -11]
```

--- Solución del sistema ---

El sistema tiene una solución única:

x1 = -0.7976

x2 = 2.7953

x3 = -0.2588

Interpretación geométrica: Los tres planos se intersecan en un único punto.

$$4x_1 + x_2 + x_3 + x_5 = 6$$

$$-x_1 - 3x_2 + x_3 + x_4 = 6$$

$$2x_1 + x_2 + 5x_3 - x_4 - x_5 = 6$$

$$-x_1 - x_2 - x_3 + 4x_4 = 6$$

$$2x_2 - x_3 + x_4 + x_5 = 6$$

```
In [5]: # Definimos la matriz y el vector del problema
A = np.array([
    [ 4, 1, 1, 0, 1],
    [-1, -3, 1, 1, 0],
    [ 2, 1, 5, -1, -1],
    [-1, -1, -1, 4, 0],
    [ 0, 2, -1, 1, 1]
])

b = np.array([6, 6, 6, 6, 6])

# Resolvemos el sistema
solucion = resolver_sistema_lineal_3d(A, b)

# Imprimimos la solución y la interpretación geométrica
print("La matriz de coeficientes es:")
print(A)
print("\nEl vector de constantes es:")
print(b)

print("\n--- Solución del sistema ---")
if isinstance(solucion, np.ndarray):
    print("El sistema tiene una solución única:")
    print(f"x1 = {solucion[0]:.4f}")
    print(f"x2 = {solucion[1]:.4f}")
    print(f"x3 = {solucion[2]:.4f}")
    print("\nInterpretación geométrica: Los tres planos se intersecan en un únic
else:
```

```
print(solucion)
print("\nInterpretación geométrica: infinitas soluciones o no tienen un punt
```

La matriz de coeficientes es:

```
[[ 4  1  1  0  1]
 [-1 -3  1  1  0]
 [ 2  1  5 -1 -1]
 [-1 -1 -1  4  0]
 [ 0  2 -1  1  1]]
```

El vector de constantes es:

```
[6 6 6 6 6]
```

--- Solución del sistema ---

El sistema tiene una solución única:

```
x1 = -1.1150
```

```
x2 = 0.2124
```

```
x3 = 3.3982
```

Interpretación geométrica: Los tres planos se intersecan en un único punto.

Ejercicio 2

Repita el ejercicio 1 usando el método de Gauss-Siede

```
In [11]: import numpy as np

def gauss_seidel_solver(A, b, initial_guess, tolerance=1e-6, max_iterations=200)
    """
    Resuelve un sistema de ecuaciones lineales Ax = b usando el método de Gauss-

    Args:
        A (np.array): La matriz de coeficientes.
        b (np.array): El vector de constantes.
        initial_guess (np.array): El vector de valores iniciales para la solució
        tolerance (float): El umbral de convergencia.
        max_iterations (int): El número máximo de iteraciones.

    Returns:
        np.array: El vector de solución aproximada.
        tuple: (mensaje de error, None) si no converge.
    """
    n = len(b)
    x = initial_guess.copy()

    for k in range(max_iterations):
        x_new = x.copy()
        for i in range(n):
            sum_val = 0
            for j in range(n):
                if i != j:
                    sum_val += A[i, j] * x_new[j]

            # Evita la división por cero
            if A[i, i] == 0:
                return "Error: la matriz no tiene diagonal no nula. El método no

        x_new[i] = (b[i] - sum_val) / A[i, i]
```

```

# Criterio de parada basado en la norma de la diferencia entre iteracion
if np.linalg.norm(x_new - x) < tolerance:
    print(f"Convergencia alcanzada en {k+1} iteraciones.")
    return x_new

x = x_new

return "No se alcanzó la convergencia dentro del número máximo de iteraciones"

```

$$3x_1 - x_2 + x_3 = 1$$

$$3x_1 + 6x_2 + 2x_3 = 0$$

$$3x_1 + 3x_2 + 7x_3 = 4$$

```

In [12]: print("--- Resolviendo sistema a. ---")
A_a = np.array([
    [3.0, -1.0, 1.0],
    [3.0, 6.0, 2.0],
    [3.0, 3.0, 7.0]
])
b_a = np.array([1.0, 0.0, 4.0])
solucion_a = gauss_seidel_solver(A_a, b_a, initial_guess=np.zeros(3))
if isinstance(solucion_a, np.ndarray):
    print("Solución para el sistema a:")
    print(f"x1 = {solucion_a[0]:.4f}, x2 = {solucion_a[1]:.4f}, x3 = {solucion_a[2]:.4f}")
else:
    print(solucion_a)

```

```

--- Resolviendo sistema a. ---
Convergencia alcanzada en 11 iteraciones.
Solución para el sistema a:
x1 = 0.0351, x2 = -0.2368, x3 = 0.6579

```

$$10x_1 - x_2 = 9$$

$$-x_1 + 10x_2 - 2x_3 = 7$$

$$-2x_2 + 10x_3 = 6$$

```

In [13]: print("--- Resolviendo sistema b. ---")
A_b = np.array([
    [10.0, -1.0, 0.0],
    [-1.0, 10.0, -2.0],
    [0.0, -2.0, 10.0]
])
b_b = np.array([9.0, 7.0, 6.0])
solucion_b = gauss_seidel_solver(A_b, b_b, initial_guess=np.zeros(3))
if isinstance(solucion_b, np.ndarray):
    print("Solución para el sistema b:")
    print(f"x1 = {solucion_b[0]:.4f}, x2 = {solucion_b[1]:.4f}, x3 = {solucion_b[2]:.4f}")
else:
    print(solucion_b)

```

--- Resolviendo sistema b. ---
 Convergencia alcanzada en 7 iteraciones.
 Solución para el sistema b:
 $x_1 = 0.9958$, $x_2 = 0.9579$, $x_3 = 0.7916$

$$10x_1 + 5x_2 = 6$$

$$5x_1 + 10x_2 - 4x_3 = 25$$

$$-4x_2 + 8x_3 - x_4 = -11$$

$$-x_3 + 5x_4 = -11$$

```
In [14]: print("--- Resolviendo sistema c. ---")
A_c = np.array([
    [10.0, 5.0, 0.0, 0.0],
    [5.0, 10.0, -4.0, 0.0],
    [0.0, -4.0, 8.0, -1.0],
    [0.0, 0.0, -1.0, 5.0]
])
b_c = np.array([6.0, 25.0, -11.0, -11.0])
solucion_c = gauss_seidel_solver(A_c, b_c, initial_guess=np.zeros(4))
if isinstance(solucion_c, np.ndarray):
    print("Solución para el sistema c:")
    print(f"x1 = {solucion_c[0]:.4f}, x2 = {solucion_c[1]:.4f}, x3 = {solucion_c[2]:.4f}, x4 = {solucion_c[3]:.4f}")
else:
    print(solucion_c)
```

--- Resolviendo sistema c. ---
 Convergencia alcanzada en 19 iteraciones.
 Solución para el sistema c:
 $x_1 = -0.7976$, $x_2 = 2.7953$, $x_3 = -0.2588$, $x_4 = -2.2518$

$$4x_1 + x_2 + x_3 + x_5 = 6$$

$$-x_1 - 3x_2 + x_3 + x_4 = 6$$

$$2x_1 + x_2 + 5x_3 - x_4 - x_5 = 6$$

$$-x_1 - x_2 - x_3 + 4x_4 = 6$$

$$2x_2 - x_3 + x_4 + x_5 = 6$$

```
In [15]: print("--- Resolviendo sistema d. ---")
A_d = np.array([
    [4.0, 1.0, 1.0, 0.0, 1.0],
    [-1.0, -3.0, 1.0, 1.0, 0.0],
    [2.0, 1.0, 5.0, -1.0, -1.0],
    [-1.0, -1.0, -1.0, 4.0, 0.0],
    [0.0, 2.0, -1.0, 1.0, 1.0]
])
b_d = np.array([6.0, 6.0, 6.0, 6.0, 6.0])
solucion_d = gauss_seidel_solver(A_d, b_d, initial_guess=np.zeros(5))
if isinstance(solucion_d, np.ndarray):
    print("Solución para el sistema d:")
    print(f"x1 = {solucion_d[0]:.4f}, x2 = {solucion_d[1]:.4f}, x3 = {solucion_d[2]:.4f}, x4 = {solucion_d[3]:.4f}, x5 = {solucion_d[4]:.4f}")
else:
    print(solucion_d)
```

```
else:
    print(solucion_d)
```

--- Resolviendo sistema d. ---

Convergencia alcanzada en 25 iteraciones.

Solución para el sistema d:

x1 = -1.1150, x2 = 0.2124, x3 = 3.3982, x4 = 2.1239, x5 = 6.8496

Ejercicio 3

Utilice el método de Jacobi para resolver los sistemas lineales en el ejercicio 1, con TOL = 10-3.

```
In [16]: import numpy as np

def jacobi_solver(A, b, initial_guess, tolerance=1e-3, max_iterations=200):
    """
    Resuelve un sistema de ecuaciones lineales Ax = b usando el método de Jacobi

    Args:
        A (np.array): La matriz de coeficientes.
        b (np.array): El vector de constantes.
        initial_guess (np.array): El vector de valores iniciales para la solución
        tolerance (float): El umbral de convergencia.
        max_iterations (int): El número máximo de iteraciones.

    Returns:
        np.array: El vector de solución aproximada.
        tuple: (mensaje de error, None) si no converge.
    """
    n = len(b)
    x = initial_guess.copy()

    for k in range(max_iterations):
        x_new = np.zeros(n)
        for i in range(n):
            sum_val = 0
            for j in range(n):
                if i != j:
                    sum_val += A[i, j] * x[j]

            # Evita la división por cero
            if A[i, i] == 0:
                return "Error: la matriz no tiene diagonal no nula. El método no"

            x_new[i] = (b[i] - sum_val) / A[i, i]

        # Criterio de parada basado en la norma de la diferencia entre iteracion
        if np.linalg.norm(x_new - x) < tolerance:
            print(f"Convergencia alcanzada en {k+1} iteraciones.")
            return x_new

        x = x_new

    return "No se alcanzó la convergencia dentro del número máximo de iteraciones"

# --- Ejemplos de uso para cada sistema ---
```



```

# a. Sistema 3x3
print("--- Resolviendo sistema a. ---")
A_a = np.array([
    [3.0, -1.0, 1.0],
    [3.0, 6.0, 2.0],
    [3.0, 3.0, 7.0]
])
b_a = np.array([1.0, 0.0, 4.0])
solucion_a = jacobi_solver(A_a, b_a, initial_guess=np.zeros(3), tolerance=1e-3)
if isinstance(solucion_a, np.ndarray):
    print("Solución para el sistema a:")
    print(f"x1 = {solucion_a[0]:.4f}, x2 = {solucion_a[1]:.4f}, x3 = {solucion_a[2]:.4f}")
else:
    print(solucion_a)

# b. Sistema 3x3
print("--- Resolviendo sistema b. ---")
A_b = np.array([
    [10.0, -1.0, 0.0],
    [-1.0, 10.0, -2.0],
    [0.0, -2.0, 10.0]
])
b_b = np.array([9.0, 7.0, 6.0])
solucion_b = jacobi_solver(A_b, b_b, initial_guess=np.zeros(3), tolerance=1e-3)
if isinstance(solucion_b, np.ndarray):
    print("Solución para el sistema b:")
    print(f"x1 = {solucion_b[0]:.4f}, x2 = {solucion_b[1]:.4f}, x3 = {solucion_b[2]:.4f}")
else:
    print(solucion_b)

# c. Sistema 4x4
print("--- Resolviendo sistema c. ---")
A_c = np.array([
    [10.0, 5.0, 0.0, 0.0],
    [5.0, 10.0, -4.0, 0.0],
    [0.0, -4.0, 8.0, -1.0],
    [0.0, 0.0, -1.0, 5.0]
])
b_c = np.array([6.0, 25.0, -11.0, -11.0])
solucion_c = jacobi_solver(A_c, b_c, initial_guess=np.zeros(4), tolerance=1e-3)
if isinstance(solucion_c, np.ndarray):
    print("Solución para el sistema c:")
    print(f"x1 = {solucion_c[0]:.4f}, x2 = {solucion_c[1]:.4f}, x3 = {solucion_c[2]:.4f}, x4 = {solucion_c[3]:.4f}")
else:
    print(solucion_c)

# d. Sistema 5x5
print("--- Resolviendo sistema d. ---")
A_d = np.array([
    [4.0, 1.0, 1.0, 0.0, 1.0],
    [-1.0, -3.0, 1.0, 1.0, 0.0],
    [2.0, 1.0, 5.0, -1.0, -1.0],
    [-1.0, -1.0, -1.0, 4.0, 0.0],
    [0.0, 2.0, -1.0, 1.0, 1.0]
])
b_d = np.array([6.0, 6.0, 6.0, 6.0, 6.0])
solucion_d = jacobi_solver(A_d, b_d, initial_guess=np.zeros(5), tolerance=1e-3)

```

```

if isinstance(solucion_d, np.ndarray):
    print("Solución para el sistema d:")
    print(f"x1 = {solucion_d[0]:.4f}, x2 = {solucion_d[1]:.4f}, x3 = {solucion_d[2]:.4f}")
else:
    print(solucion_d)

```

--- Resolviendo sistema a. ---

Convergencia alcanzada en 10 iteraciones.

Solución para el sistema a:

x1 = 0.0351, x2 = -0.2369, x3 = 0.6578

--- Resolviendo sistema b. ---

Convergencia alcanzada en 6 iteraciones.

Solución para el sistema b:

x1 = 0.9957, x2 = 0.9578, x3 = 0.7914

--- Resolviendo sistema c. ---

Convergencia alcanzada en 22 iteraciones.

Solución para el sistema c:

x1 = -0.7976, x2 = 2.7948, x3 = -0.2589, x4 = -2.2519

--- Resolviendo sistema d. ---

Convergencia alcanzada en 27 iteraciones.

Solución para el sistema d:

x1 = -1.1148, x2 = 0.2126, x3 = 3.3979, x4 = 2.1240, x5 = 6.8480

Ejercicio 4

Utilice el método de Gauss-Siedel para resolver los sistemas lineales en el ejercicio 1, con $TOL = 10^{-3}$.

```

In [18]: import numpy as np

def gauss_seidel_solver(A, b, initial_guess, tolerance=1e-3, max_iterations=200)
    """
    Resuelve un sistema de ecuaciones lineales Ax = b usando el método de Gauss-
    Siedel.

    Args:
        A (np.array): La matriz de coeficientes.
        b (np.array): El vector de constantes.
        initial_guess (np.array): El vector de valores iniciales para la solución.
        tolerance (float): El umbral de convergencia.
        max_iterations (int): El número máximo de iteraciones.

    Returns:
        np.array: El vector de solución aproximada.
        tuple: (mensaje de error, None) si no converge.
    """
    n = len(b)
    x = initial_guess.copy()

    for k in range(max_iterations):
        x_new = x.copy()
        for i in range(n):
            sum_val = 0
            for j in range(n):
                if i != j:
                    sum_val += A[i, j] * x_new[j]
            x_new[i] = (b[i] - sum_val) / A[i, i]

    return x_new

```

```

        # Evita la división por cero
        if A[i, i] == 0:
            return "Error: la matriz no tiene diagonal no nula. El método no

        x_new[i] = (b[i] - sum_val) / A[i, i]

    # Criterio de parada basado en la norma de la diferencia entre iteracion
    if np.linalg.norm(x_new - x) < tolerance:
        print(f"Convergencia alcanzada en {k+1} iteraciones.")
        return x_new

    x = x_new

    return "No se alcanzó la convergencia dentro del número máximo de iteracione

# --- Ejemplos de uso para cada sistema ---

# a. Sistema 3x3
print("--- Resolviendo sistema a. ---")
A_a = np.array([
    [3.0, -1.0, 1.0],
    [3.0, 6.0, 2.0],
    [3.0, 3.0, 7.0]
])
b_a = np.array([1.0, 0.0, 4.0])
solucion_a = gauss_seidel_solver(A_a, b_a, initial_guess=np.zeros(3), tolerance=
if isinstance(solucion_a, np.ndarray):
    print("Solución para el sistema a:")
    print(f"x1 = {solucion_a[0]:.4f}, x2 = {solucion_a[1]:.4f}, x3 = {solucion_a
else:
    print(solucion_a)

# b. Sistema 3x3
print("--- Resolviendo sistema b. ---")
A_b = np.array([
    [10.0, -1.0, 0.0],
    [-1.0, 10.0, -2.0],
    [0.0, -2.0, 10.0]
])
b_b = np.array([9.0, 7.0, 6.0])
solucion_b = gauss_seidel_solver(A_b, b_b, initial_guess=np.zeros(3), tolerance=
if isinstance(solucion_b, np.ndarray):
    print("Solución para el sistema b:")
    print(f"x1 = {solucion_b[0]:.4f}, x2 = {solucion_b[1]:.4f}, x3 = {solucion_b
else:
    print(solucion_b)

# c. Sistema 4x4
print("--- Resolviendo sistema c. ---")
A_c = np.array([
    [10.0, 5.0, 0.0, 0.0],
    [5.0, 10.0, -4.0, 0.0],
    [0.0, -4.0, 8.0, -1.0],
    [0.0, 0.0, -1.0, 5.0]
])
b_c = np.array([6.0, 25.0, -11.0, -11.0])
solucion_c = gauss_seidel_solver(A_c, b_c, initial_guess=np.zeros(4), tolerance=

```

```

if isinstance(solucion_c, np.ndarray):
    print("Solución para el sistema c:")
    print(f"x1 = {solucion_c[0]:.4f}, x2 = {solucion_c[1]:.4f}, x3 = {solucion_c[2]:.4f}")
else:
    print(solucion_c)

# d. Sistema 5x5
print("--- Resolviendo sistema d. ---")
A_d = np.array([
    [4.0, 1.0, 1.0, 0.0, 1.0],
    [-1.0, -3.0, 1.0, 1.0, 0.0],
    [2.0, 1.0, 5.0, -1.0, -1.0],
    [-1.0, -1.0, -1.0, 4.0, 0.0],
    [0.0, 2.0, -1.0, 1.0, 1.0]
])
b_d = np.array([6.0, 6.0, 6.0, 6.0, 6.0])
solucion_d = gauss_seidel_solver(A_d, b_d, initial_guess=np.zeros(5), tolerance=1e-6)
if isinstance(solucion_d, np.ndarray):
    print("Solución para el sistema d:")
    print(f"x1 = {solucion_d[0]:.4f}, x2 = {solucion_d[1]:.4f}, x3 = {solucion_d[2]:.4f}, x4 = {solucion_d[3]:.4f}, x5 = {solucion_d[4]:.4f}")
else:
    print(solucion_d)

```

--- Resolviendo sistema a. ---

Convergencia alcanzada en 6 iteraciones.

Solución para el sistema a:

x1 = 0.0354, x2 = -0.2368, x3 = 0.6578

--- Resolviendo sistema b. ---

Convergencia alcanzada en 4 iteraciones.

Solución para el sistema b:

x1 = 0.9957, x2 = 0.9579, x3 = 0.7916

--- Resolviendo sistema c. ---

Convergencia alcanzada en 10 iteraciones.

Solución para el sistema c:

x1 = -0.7973, x2 = 2.7950, x3 = -0.2590, x4 = -2.2518

--- Resolviendo sistema d. ---

Convergencia alcanzada en 15 iteraciones.

Solución para el sistema d:

x1 = -1.1149, x2 = 0.2122, x3 = 3.3981, x4 = 2.1239, x5 = 6.8498

Ejercicio 5

Solucione el sistema lineal:

$$\begin{aligned}
 2x_1 - x_2 + x_3 &= -1, \\
 2x_1 + 2x_2 + 2x_3 &= 4, \\
 -x_1 - x_2 + 2x_3 &= -5,
 \end{aligned}$$

tiene la solución (1, 2, -1)t.

a) Muestre que el método de Jacobi con $x(0) = 0$ falla al proporcionar una buena aproximación después de 25 iteraciones.

```
In [19]: import numpy as np

# Sistema de la imagen
A = np.array([
    [ 2, -1, 1],
    [ 2, 2, 2],
    [-1, -1, 2]
], dtype=float)

b = np.array([-1, 4, -5], dtype=float)

# Solución exacta conocida (para comparar)
sol_exacta = np.array([1, 2, -1], dtype=float)

# Método de Jacobi
def jacobi(A, b, x0, iter_max=25, tol=1e-8):
    n = len(b)
    x = x0.copy()
    for it in range(iter_max):
        x_new = np.zeros_like(x)
        for i in range(n):
            s = 0
            for j in range(n):
                if j != i:
                    s += A[i,j]*x[j]
            x_new[i] = (b[i] - s) / A[i,i]
        x = x_new
        print(f"Iteración {it+1:2d}: x = {x}")
    return x

#  $x(0) = 0$ 
x0 = np.zeros(3)
resultado = jacobi(A, b, x0, iter_max=25)

print("\nResultado tras 25 iteraciones:")
print(resultado)
print("Solución exacta:", sol_exacta)
print("Error:", resultado - sol_exacta)
```

```

Iteración 1: x = [-0.5  2.  -2.5]
Iteración 2: x = [ 1.75  5.  -1.75]
Iteración 3: x = [2.875 2.   0.875]
Iteración 4: x = [ 0.0625 -1.75  -0.0625]
Iteración 5: x = [-1.34375  2.   -3.34375]
Iteración 6: x = [ 2.171875  6.6875  -2.171875]
Iteración 7: x = [3.9296875 2.   1.9296875]
Iteración 8: x = [-0.46484375 -3.859375  0.46484375]
Iteración 9: x = [-2.66210938 2.   -4.66210938]
Iteración 10: x = [ 2.83105469 9.32421875 -2.83105469]
Iteración 11: x = [5.57763672 2.   3.57763672]
Iteración 12: x = [-1.28881836 -7.15527344 1.28881836]
Iteración 13: x = [-4.7220459 2.   -6.7220459]
Iteración 14: x = [ 3.86102295 13.4440918 -3.86102295]
Iteración 15: x = [8.15255737 2.   6.15255737]
Iteración 16: x = [-2.57627869 -12.30511475 2.57627869]
Iteración 17: x = [-7.94069672 2.   -9.94069672]
Iteración 18: x = [ 5.47034836 19.88139343 -5.47034836]
Iteración 19: x = [12.1758709 2.   10.1758709]
Iteración 20: x = [-4.58793545 -20.35174179 4.58793545]
Iteración 21: x = [-12.96983862 2.   -14.96983862]
Iteración 22: x = [ 7.98491931 29.93967724 -7.98491931]
Iteración 23: x = [18.46229827 2.   16.46229827]
Iteración 24: x = [-7.73114914 -32.92459655 7.73114914]
Iteración 25: x = [-20.82787284 2.   -22.82787284]

```

Resultado tras 25 iteraciones:

```
[-20.82787284  2.   -22.82787284]
```

Solución exacta: [1. 2. -1.]

```
Error: [-21.82787284  0.   -21.82787284]
```

In [20]: `import numpy as np`

Sistema de la imagen

```
A = np.array([
    [ 2, -1,  1],
    [ 2,  2,  2],
    [-1, -1,  2]
], dtype=float)
```

```
b = np.array([-1, 4, -5], dtype=float)
```

Solución exacta (para comparar)

```
sol_exacta = np.array([1, 2, -1], dtype=float)
```

```
def gauss_seidel(A, b, x0=None, tol=1e-5, iter_max=1000):
```

```
    n = len(b)
```

```
    if x0 is None:
```

```
        x = np.zeros(n)
```

```
    else:
```

```
        x = x0.copy()
```

```
    for it in range(iter_max):
```

```
        x_old = x.copy()
```

```
        for i in range(n):
```

```
            s1 = sum(A[i, j] * x[j] for j in range(i))
```

usa valores actual

```
            s2 = sum(A[i, j] * x_old[j] for j in range(i+1, n))
```

usa valores a

```
            x[i] = (b[i] - s1 - s2) / A[i, i]
```

Norma infinita del error

```

error = np.linalg.norm(x - x_old, ord=np.inf)
print(f"Iteración {it+1:2d}: x = {x}, error = {error:.6e}")
if error < tol:
    print(f"\n✅ Convergencia alcanzada en {it+1} iteraciones.")
    return x

print("\n⚠️ No alcanzó la tolerancia después del máximo de iteraciones.")
return x

# x(0) = (0,0,0)
x0 = np.zeros(3)
resultado = gauss_seidel(A, b, x0, tol=1e-5)

print("\nResultado aproximado:", resultado)
print("Solución exacta:", sol_exacta)
print("Error final:", resultado - sol_exacta)

```

```

Iteración 1: x = [-0.5  2.5 -1.5], error = 2.500000e+00
Iteración 2: x = [ 1.5   2.   -0.75], error = 2.000000e+00
Iteración 3: x = [ 0.875  1.875 -1.125], error = 6.250000e-01
Iteración 4: x = [ 1.      2.125 -0.9375], error = 2.500000e-01
Iteración 5: x = [ 1.03125  1.90625 -1.03125], error = 2.187500e-01
Iteración 6: x = [ 0.96875  2.0625  -0.984375], error = 1.562500e-01
Iteración 7: x = [ 1.0234375  1.9609375 -1.0078125], error = 1.015625e-01
Iteración 8: x = [ 0.984375  2.0234375 -0.99609375], error = 6.250000e-02
Iteración 9: x = [ 1.00976562  1.98632812 -1.00195312], error = 3.710938e-02
Iteración 10: x = [ 0.99414062  2.0078125  -0.99902344], error = 2.148438e-02
Iteración 11: x = [ 1.00341797  1.99560547 -1.00048828], error = 1.220703e-02
Iteración 12: x = [ 0.99804688  2.00244141 -0.99975586], error = 6.835938e-03
Iteración 13: x = [ 1.00109863  1.99865723 -1.00012207], error = 3.784180e-03
Iteración 14: x = [ 0.99938965  2.00073242 -0.99993896], error = 2.075195e-03
Iteración 15: x = [ 1.00033569  1.99960327 -1.00003052], error = 1.129150e-03
Iteración 16: x = [ 0.99981689  2.00021362 -0.99998474], error = 6.103516e-04
Iteración 17: x = [ 1.00009918  1.99988556 -1.00000763], error = 3.280640e-04
Iteración 18: x = [ 0.99994659  2.00006104 -0.99999619], error = 1.754761e-04
Iteración 19: x = [ 1.00002861  1.99996758 -1.00000191], error = 9.346008e-05
Iteración 20: x = [ 0.99998474  2.00001717 -0.99999905], error = 4.959106e-05
Iteración 21: x = [ 1.00000811  1.99999094 -1.00000048], error = 2.622604e-05
Iteración 22: x = [ 0.99999571  2.00000477 -0.99999976], error = 1.382828e-05
Iteración 23: x = [ 1.00000226  1.9999975  -1.00000012], error = 7.271767e-06

```

✅ Convergencia alcanzada en 23 iteraciones.

Resultado aproximado: [1.00000226 1.9999975 -1.00000012]
 Solución exacta: [1. 2. -1.]
 Error final: [2.26497650e-06 -2.50339508e-06 -1.19209290e-07]

Ejercicio 6

El sistema lineal:

$$\begin{aligned}
 x_1 & - x_3 = 0.2, \\
 -\frac{1}{2}x_1 + x_2 - \frac{1}{4}x_3 & = -1.425, \\
 x_1 - \frac{1}{2}x_2 + x_3 & = 2,
 \end{aligned}$$

tiene la solución (0.9, -0.8, 0.7)

a) La matriz de coeficientes

$$A = \begin{bmatrix} 1 & 0 & -1 \\ -\frac{1}{2} & 1 & -\frac{1}{4} \\ 1 & -\frac{1}{2} & 1 \end{bmatrix}$$

tiene diagonal estrictamente dominante?

```
In [23]: print(abs(1) > 0 + (-1))
print(abs(1) > (-1/2) + (-1/4))
print(abs(1) > (-1/2) + (1))
```

True

True

True

Si la matriz A tiene una diagonal estrictamente dominante ya que el valor absoluto de la diagonal principal es mayor que el valor absoluto de cualquier otra entrada en la matriz.

b) Utilice el método iterativo de Gauss-Siedel para aproximar la solución para el sistema lineal con una tolerancia de 1022 y un máximo de 300 iteraciones.

```
In [25]: import numpy as np

# Matriz A
A = np.array([
    [ 1, 0, -1 ],
    [-0.5, 1, -0.25 ],
    [ 1, -0.5, 1 ]
], dtype=float)

# Vector b
b = np.array([0.2, -1.425, 2.0], dtype=float)

def gauss_seidel(A, b, x0=None, tol=1e-22, iter_max=300):
    n = len(b)
    # vector inicial
    if x0 is None:
        x = np.zeros(n)
    else:
        x = x0.copy()

    for it in range(iter_max):
        x_old = x.copy()
        for i in range(n):
            s1 = sum(A[i, j] * x[j] for j in range(i))
            s2 = sum(A[i, j] * x_old[j] for j in range(i+1, n))
            x[i] = (b[i] - s1 - s2) / A[i, i]
        # norma infinito del cambio
        error = np.linalg.norm(x - x_old, ord=np.inf)
        print(f"Iteración {it+1:3d}: x = {x}, error = {error:.3e}")
        if error < tol:
            print(f"\n✅ Convergencia alcanzada en {it+1} iteraciones.")
            return x
```



```
    print("\n⚠️ No se alcanzó la tolerancia después de 300 iteraciones.")
    return x

# Ejecutar con x0 = (0,0,0)
x0 = np.zeros(3)
resultado = gauss_seidel(A, b, x0, tol=1e-22, iter_max=300)

print("\nResultado aproximado:", resultado)
```

```

Iteración 1: x = [ 0.2    -1.325    1.1375], error = 1.325e+00
Iteración 2: x = [ 1.3375   -0.471875   0.4265625], error = 1.137e+00
Iteración 3: x = [ 0.6265625 -1.00507813  0.87089844], error = 7.109e-01
Iteración 4: x = [ 1.07089844 -0.67182617  0.59318848], error = 4.443e-01
Iteración 5: x = [ 0.79318848 -0.88010864  0.7667572 ], error = 2.777e-01
Iteración 6: x = [ 0.9667572  -0.7499321   0.65827675], error = 1.736e-01
Iteración 7: x = [ 0.85827675 -0.83129244  0.72607703], error = 1.085e-01
Iteración 8: x = [ 0.92607703 -0.78044223  0.68370185], error = 6.780e-02
Iteración 9: x = [ 0.88370185 -0.81222361  0.71018634], error = 4.238e-02
Iteración 10: x = [ 0.91018634 -0.79236024  0.69363354], error = 2.648e-02
Iteración 11: x = [ 0.89363354 -0.80477485  0.70397904], error = 1.655e-02
Iteración 12: x = [ 0.90397904 -0.79701572  0.6975131 ], error = 1.035e-02
Iteración 13: x = [ 0.8975131  -0.80186517  0.70155431], error = 6.466e-03
Iteración 14: x = [ 0.90155431 -0.79883427  0.69902855], error = 4.041e-03
Iteración 15: x = [ 0.89902855 -0.80072858  0.70060715], error = 2.526e-03
Iteración 16: x = [ 0.90060715 -0.79954464  0.69962053], error = 1.579e-03
Iteración 17: x = [ 0.89962053 -0.8002846   0.70023717], error = 9.866e-04
Iteración 18: x = [ 0.90023717 -0.79982212  0.69985177], error = 6.166e-04
Iteración 19: x = [ 0.89985177 -0.80011117  0.70009264], error = 3.854e-04
Iteración 20: x = [ 0.90009264 -0.79993052  0.6999421 ], error = 2.409e-04
Iteración 21: x = [ 0.8999421  -0.80004343  0.70003619], error = 1.505e-04
Iteración 22: x = [ 0.90003619 -0.79997286  0.69997738], error = 9.409e-05
Iteración 23: x = [ 0.89997738 -0.80001696  0.70001414], error = 5.881e-05
Iteración 24: x = [ 0.90001414 -0.7999894   0.69999116], error = 3.675e-05
Iteración 25: x = [ 0.89999116 -0.80000663  0.70000552], error = 2.297e-05
Iteración 26: x = [ 0.90000552 -0.79999586  0.69999655], error = 1.436e-05
Iteración 27: x = [ 0.89999655 -0.80000259  0.70000216], error = 8.973e-06
Iteración 28: x = [ 0.90000216 -0.79999838  0.69999865], error = 5.608e-06
Iteración 29: x = [ 0.89999865 -0.80000101  0.70000084], error = 3.505e-06
Iteración 30: x = [ 0.90000084 -0.79999937  0.69999947], error = 2.191e-06
Iteración 31: x = [ 0.89999947 -0.80000039  0.70000033], error = 1.369e-06
Iteración 32: x = [ 0.90000033 -0.79999975  0.69999979], error = 8.558e-07
Iteración 33: x = [ 0.89999979 -0.80000015  0.70000013], error = 5.348e-07
Iteración 34: x = [ 0.90000013 -0.7999999   0.69999992], error = 3.343e-07
Iteración 35: x = [ 0.89999992 -0.80000006  0.70000005], error = 2.089e-07
Iteración 36: x = [ 0.90000005 -0.79999996  0.69999997], error = 1.306e-07
Iteración 37: x = [ 0.89999997 -0.80000002  0.70000002], error = 8.161e-08
Iteración 38: x = [ 0.90000002 -0.79999999  0.69999999], error = 5.101e-08
Iteración 39: x = [ 0.89999999 -0.80000001  0.70000001], error = 3.188e-08
Iteración 40: x = [ 0.90000001 -0.79999999  0.7          ], error = 1.992e-08
Iteración 41: x = [ 0.9 -0.8  0.7], error = 1.245e-08
Iteración 42: x = [ 0.9 -0.8  0.7], error = 7.783e-09
Iteración 43: x = [ 0.9 -0.8  0.7], error = 4.864e-09
Iteración 44: x = [ 0.9 -0.8  0.7], error = 3.040e-09
Iteración 45: x = [ 0.9 -0.8  0.7], error = 1.900e-09
Iteración 46: x = [ 0.9 -0.8  0.7], error = 1.188e-09
Iteración 47: x = [ 0.9 -0.8  0.7], error = 7.423e-10
Iteración 48: x = [ 0.9 -0.8  0.7], error = 4.639e-10
Iteración 49: x = [ 0.9 -0.8  0.7], error = 2.899e-10
Iteración 50: x = [ 0.9 -0.8  0.7], error = 1.812e-10
Iteración 51: x = [ 0.9 -0.8  0.7], error = 1.133e-10
Iteración 52: x = [ 0.9 -0.8  0.7], error = 7.079e-11
Iteración 53: x = [ 0.9 -0.8  0.7], error = 4.424e-11
Iteración 54: x = [ 0.9 -0.8  0.7], error = 2.765e-11
Iteración 55: x = [ 0.9 -0.8  0.7], error = 1.728e-11
Iteración 56: x = [ 0.9 -0.8  0.7], error = 1.080e-11
Iteración 57: x = [ 0.9 -0.8  0.7], error = 6.751e-12
Iteración 58: x = [ 0.9 -0.8  0.7], error = 4.219e-12
Iteración 59: x = [ 0.9 -0.8  0.7], error = 2.637e-12
Iteración 60: x = [ 0.9 -0.8  0.7], error = 1.648e-12

```

Iteración 61: $x = [0.9 \ -0.8 \ 0.7]$, error = $1.030e-12$
Iteración 62: $x = [0.9 \ -0.8 \ 0.7]$, error = $6.437e-13$
Iteración 63: $x = [0.9 \ -0.8 \ 0.7]$, error = $4.023e-13$
Iteración 64: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.516e-13$
Iteración 65: $x = [0.9 \ -0.8 \ 0.7]$, error = $1.572e-13$
Iteración 66: $x = [0.9 \ -0.8 \ 0.7]$, error = $9.837e-14$
Iteración 67: $x = [0.9 \ -0.8 \ 0.7]$, error = $6.173e-14$
Iteración 68: $x = [0.9 \ -0.8 \ 0.7]$, error = $3.864e-14$
Iteración 69: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.398e-14$
Iteración 70: $x = [0.9 \ -0.8 \ 0.7]$, error = $1.488e-14$
Iteración 71: $x = [0.9 \ -0.8 \ 0.7]$, error = $9.326e-15$
Iteración 72: $x = [0.9 \ -0.8 \ 0.7]$, error = $5.995e-15$
Iteración 73: $x = [0.9 \ -0.8 \ 0.7]$, error = $3.997e-15$
Iteración 74: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.665e-15$
Iteración 75: $x = [0.9 \ -0.8 \ 0.7]$, error = $1.776e-15$
Iteración 76: $x = [0.9 \ -0.8 \ 0.7]$, error = $1.110e-15$
Iteración 77: $x = [0.9 \ -0.8 \ 0.7]$, error = $6.661e-16$
Iteración 78: $x = [0.9 \ -0.8 \ 0.7]$, error = $4.441e-16$
Iteración 79: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 80: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 81: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 82: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 83: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 84: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 85: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 86: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 87: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 88: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 89: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 90: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 91: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 92: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 93: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 94: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 95: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 96: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 97: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 98: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 99: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 100: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 101: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 102: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 103: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 104: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 105: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 106: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 107: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 108: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 109: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 110: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 111: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 112: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 113: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 114: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 115: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 116: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 117: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 118: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 119: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$
Iteración 120: $x = [0.9 \ -0.8 \ 0.7]$, error = $2.220e-16$

[illegible]

[illegible]

[illegible]

⚠️ No se alcanzó la tolerancia después de 300 iteraciones.

Resultado aproximado: [0.9 -0.8 0.7]

No alcanza esa tolerancia porque es demasiado estricta para la precisión de la máquina y además la matriz no es ideal para convergencia rápida.

c) . ¿Qué pasa en la parte b) cuando el sistema cambia por el siguiente?

El método de Gauss-Seidel no alcanza la tolerancia de 10^{-22} después de 300 iteraciones. Esto sucede porque:

La matriz no es diagonalmente dominante ni garantiza convergencia rápida.

La tolerancia 10^{-22} es demasiado estricta para la precisión de la máquina (doble precisión $\approx 10^{-16}$).

Ejercicio 7

Repita el ejercicio 11 usando el método de Jacobi.

```
In [26]: import numpy as np

# Matriz A
A = np.array([
    [ 1, 0, -1 ],
    [-0.5, 1, -0.25 ],
    [ 1, -0.5, 1 ]
], dtype=float)

# Vector b
b = np.array([0.2, -1.425, 2.0], dtype=float)

def jacobi(A, b, x0=None, tol=1e-22, iter_max=300):
    n = len(b)
    if x0 is None:
        x = np.zeros(n)
    else:
        x = x0.copy()

    for it in range(iter_max):
        x_new = np.zeros_like(x)
        for i in range(n):
            s = sum(A[i,j]*x[j] for j in range(n) if j != i)
            x_new[i] = (b[i] - s) / A[i,i]

        # error como norma infinito
        error = np.linalg.norm(x_new - x, ord=np.inf)
        print(f"Iteración {it+1:3d}: x = {x_new}, error = {error:.3e}")

        if error < tol:
            print(f"\n✅ Convergencia alcanzada en {it+1} iteraciones.")
            return x_new
        x = x_new

    print("\n⚠️ No se alcanzó la tolerancia después de 300 iteraciones.")
```

```
    return x

# Ejecutar Jacobi
x0 = np.zeros(3)
resultado = jacobi(A, b, x0, tol=1e-22, iter_max=300)

print("\nResultado aproximado:", resultado)
```



```

Iteración 1: x = [ 0.2   -1.425  2.   ], error = 2.000e+00
Iteración 2: x = [ 2.2    -0.825  1.0875], error = 2.000e+00
Iteración 3: x = [ 1.2875  -0.053125 -0.6125 ], error = 1.700e+00
Iteración 4: x = [-0.4125   -0.934375   0.6859375], error = 1.700e+00
Iteración 5: x = [ 0.8859375  -1.45976563  1.9453125 ], error = 1.298e+00
Iteración 6: x = [ 2.1453125  -0.49570312  0.38417969], error = 1.561e+00
Iteración 7: x = [ 0.58417969 -0.25629883 -0.39316406], error = 1.561e+00
Iteración 8: x = [-0.19316406 -1.23120117  1.2876709 ], error = 1.681e+00
Iteración 9: x = [ 1.4876709  -1.19966431  1.57756348], error = 1.681e+00
Iteración 10: x = [ 1.77756348 -0.28677368 -0.08750305], error = 1.665e+00
Iteración 11: x = [ 0.11249695 -0.55809402  0.07904968], error = 1.665e+00
Iteración 12: x = [ 0.27904968 -1.34898911  1.60845604], error = 1.529e+00
Iteración 13: x = [ 1.80845604 -0.88336115  1.04645576], error = 1.529e+00
Iteración 14: x = [ 1.24645576 -0.25915804 -0.25013661], error = 1.297e+00
Iteración 15: x = [-0.05013661 -0.86430627  0.62396522], error = 1.297e+00
Iteración 16: x = [ 0.82396522 -1.294077   1.61798348], error = 9.940e-01
Iteración 17: x = [ 1.81798348 -0.60852152  0.52899628], error = 1.089e+00
Iteración 18: x = [ 0.72899628 -0.38375919 -0.12224424], error = 1.089e+00
Iteración 19: x = [ 0.07775576 -1.09106292  1.07912412], error = 1.201e+00
Iteración 20: x = [ 1.27912412 -1.11634109  1.37671278], error = 1.201e+00
Iteración 21: x = [ 1.57671278 -0.44125974  0.16270533], error = 1.214e+00
Iteración 22: x = [ 0.36270533 -0.59596728  0.20265735], error = 1.214e+00
Iteración 23: x = [ 0.40265735 -1.192983   1.33931103], error = 1.137e+00
Iteración 24: x = [ 1.53931103 -0.88884357  1.00085115], error = 1.137e+00
Iteración 25: x = [ 1.20085115 -0.4051317   0.01626719], error = 9.846e-01
Iteración 26: x = [ 0.21626719 -0.82050763  0.596583   ], error = 9.846e-01
Iteración 27: x = [ 0.796583   -1.16772066  1.373479   ], error = 7.769e-01
Iteración 28: x = [ 1.573479   -0.68333875  0.61955667], error = 7.769e-01
Iteración 29: x = [ 0.81955667 -0.48337133  0.08485162], error = 7.539e-01
Iteración 30: x = [ 0.28485162 -0.99400876  0.93875766], error = 8.539e-01
Iteración 31: x = [ 1.13875766 -1.04788477  1.218144   ], error = 8.539e-01
Iteración 32: x = [ 1.418144   -0.55108517  0.33729995], error = 8.808e-01
Iteración 33: x = [ 0.53729995 -0.63160301  0.30631342], error = 8.808e-01
Iteración 34: x = [ 0.50631342 -1.07977167  1.14689854], error = 8.406e-01
Iteración 35: x = [ 1.34689854 -0.88511866  0.95380075], error = 8.406e-01
Iteración 36: x = [ 1.15380075 -0.51310054  0.21054213], error = 7.433e-01
Iteración 37: x = [ 0.41054213 -0.79546409  0.58964898], error = 7.433e-01
Iteración 38: x = [ 0.78964898 -1.07231669  1.19172582], error = 6.021e-01
Iteración 39: x = [ 1.39172582 -0.73224405  0.67419268], error = 6.021e-01
Iteración 40: x = [ 0.87419268 -0.56058892  0.24215215], error = 5.175e-01
Iteración 41: x = [ 0.44215215 -0.92736562  0.84551286], error = 6.034e-01
Iteración 42: x = [ 1.04551286 -0.99254571  1.09416504], error = 6.034e-01
Iteración 43: x = [ 1.29416504 -0.62870231  0.45821428], error = 6.360e-01
Iteración 44: x = [ 0.65821428 -0.66336391  0.39148381], error = 6.360e-01
Iteración 45: x = [ 0.59148381 -0.99802191  1.01010376], error = 6.186e-01
Iteración 46: x = [ 1.21010376 -0.87673215  0.90950524], error = 6.186e-01
Iteración 47: x = [ 1.10950524 -0.59257181  0.35153016], error = 5.580e-01
Iteración 48: x = [ 0.55153016 -0.78236484  0.59420886], error = 5.580e-01
Iteración 49: x = [ 0.79420886 -1.00068271  1.05728742], error = 4.631e-01
Iteración 50: x = [ 1.25728742 -0.76357372  0.70544979], error = 4.631e-01
Iteración 51: x = [ 0.90544979 -0.61999384  0.36092572], error = 3.518e-01
Iteración 52: x = [ 0.56092572 -0.88204367  0.78455329], error = 4.236e-01
Iteración 53: x = [ 0.98455329 -0.94839882  0.99805244], error = 4.236e-01
Iteración 54: x = [ 1.19805244 -0.68321025  0.5412473 ], error = 4.568e-01
Iteración 55: x = [ 0.7412473  -0.69066195  0.46034244], error = 4.568e-01
Iteración 56: x = [ 0.66034244 -0.93929074  0.91342172], error = 4.531e-01
Iteración 57: x = [ 1.11342172 -0.86647335  0.87001219], error = 4.531e-01
Iteración 58: x = [ 1.07001219 -0.65078609  0.4533416 ], error = 4.167e-01
Iteración 59: x = [ 0.6533416  -0.7766585  0.60459476], error = 4.167e-01
Iteración 60: x = [ 0.80459476 -0.94718051  0.95832914], error = 3.537e-01

```

```

Iteración 61: x = [ 1.15832914 -0.78312033 0.72181498], error = 3.537e-01
Iteración 62: x = [ 0.92181498 -0.66538168 0.45011069], error = 2.717e-01
Iteración 63: x = [ 0.65011069 -0.85156484 0.74549417], error = 2.954e-01
Iteración 64: x = [ 0.94549417 -0.91357111 0.92410689], error = 2.954e-01
Iteración 65: x = [ 1.12410689 -0.72122619 0.59772027], error = 3.264e-01
Iteración 66: x = [ 0.79772027 -0.71351649 0.51528001], error = 3.264e-01
Iteración 67: x = [ 0.71528001 -0.89731986 0.84552149], error = 3.302e-01
Iteración 68: x = [ 1.04552149 -0.85597962 0.83606006], error = 3.302e-01
Iteración 69: x = [ 1.03606006 -0.69322424 0.5264887 ], error = 3.096e-01
Iteración 70: x = [ 0.7264887 -0.7753478 0.61732782], error = 3.096e-01
Iteración 71: x = [ 0.81732782 -0.90742369 0.8858374 ], error = 2.685e-01
Iteración 72: x = [ 1.0858374 -0.79487674 0.72896033], error = 2.685e-01
Iteración 73: x = [ 0.92896033 -0.69984122 0.51672423], error = 2.122e-01
Iteración 74: x = [ 0.71672423 -0.83133878 0.72111906], error = 2.122e-01
Iteración 75: x = [ 0.92111906 -0.88635812 0.86760638], error = 2.044e-01
Iteración 76: x = [ 1.06760638 -0.74753887 0.63570188], error = 2.319e-01
Iteración 77: x = [ 0.83570188 -0.73227134 0.55862418], error = 2.319e-01
Iteración 78: x = [ 0.75862418 -0.86749301 0.79816245], error = 2.395e-01
Iteración 79: x = [ 0.99816245 -0.8461473 0.80762931], error = 2.395e-01
Iteración 80: x = [ 1.00762931 -0.72401145 0.5787639 ], error = 2.289e-01
Iteración 81: x = [ 0.7787639 -0.77649437 0.63036497], error = 2.289e-01
Iteración 82: x = [ 0.83036497 -0.87802681 0.83298891], error = 2.026e-01
Iteración 83: x = [ 1.03298891 -0.80157029 0.73062163], error = 2.026e-01
Iteración 84: x = [ 0.93062163 -0.72585014 0.56622594], error = 1.644e-01
Iteración 85: x = [ 0.76622594 -0.8181327 0.7064533 ], error = 1.644e-01
Iteración 86: x = [ 0.9064533 -0.8652737 0.82470771], error = 1.402e-01
Iteración 87: x = [ 1.02470771 -0.76559642 0.66090985], error = 1.638e-01
Iteración 88: x = [ 0.86090985 -0.74741868 0.59249408], error = 1.638e-01
Iteración 89: x = [ 0.79249408 -0.84642156 0.76538081], error = 1.729e-01
Iteración 90: x = [ 0.96538081 -0.83740776 0.78429514], error = 1.729e-01
Iteración 91: x = [ 0.98429514 -0.74623581 0.61591531], error = 1.684e-01
Iteración 92: x = [ 0.81591531 -0.7788736 0.64258695], error = 1.684e-01
Iteración 93: x = [ 0.84258695 -0.85639561 0.79464789], error = 1.521e-01
Iteración 94: x = [ 0.99464789 -0.80504455 0.72921524], error = 1.521e-01
Iteración 95: x = [ 0.92921524 -0.74537224 0.60282984], error = 1.264e-01
Iteración 96: x = [ 0.80282984 -0.80968492 0.69809864], error = 1.264e-01
Iteración 97: x = [ 0.89809864 -0.84906042 0.7923277 ], error = 9.527e-02
Iteración 98: x = [ 0.9923277 -0.77786876 0.67737115], error = 1.150e-01
Iteración 99: x = [ 0.87737115 -0.75949336 0.61873792], error = 1.150e-01
Iteración 100: x = [ 0.81873792 -0.83162994 0.74288217], error = 1.241e-01
Iteración 101: x = [ 0.94288217 -0.8299105 0.76544711], error = 1.241e-01
Iteración 102: x = [ 0.96544711 -0.76219714 0.64216258], error = 1.233e-01
Iteración 103: x = [ 0.84216258 -0.7817358 0.65345432], error = 1.233e-01
Iteración 104: x = [ 0.85345432 -0.84055513 0.76696952], error = 1.135e-01
Iteración 105: x = [ 0.96696952 -0.80653046 0.72626812], error = 1.135e-01
Iteración 106: x = [ 0.92626812 -0.75994821 0.62976525], error = 9.650e-02
Iteración 107: x = [ 0.82976525 -0.80442463 0.69375778], error = 9.650e-02
Iteración 108: x = [ 0.89375778 -0.83667793 0.76802243], error = 7.426e-02
Iteración 109: x = [ 0.96802243 -0.7861155 0.68790326], error = 8.012e-02
Iteración 110: x = [ 0.88790326 -0.76901297 0.63891982], error = 8.012e-02
Iteración 111: x = [ 0.83891982 -0.82131842 0.72759026], error = 8.867e-02
Iteración 112: x = [ 0.92759026 -0.82364253 0.75042098], error = 8.867e-02
Iteración 113: x = [ 0.95042098 -0.77359963 0.66058848], error = 8.983e-02
Iteración 114: x = [ 0.86058848 -0.78464239 0.66277921], error = 8.983e-02
Iteración 115: x = [ 0.86277921 -0.82901096 0.74709033], error = 8.431e-02
Iteración 116: x = [ 0.94709033 -0.80683781 0.72271531], error = 8.431e-02
Iteración 117: x = [ 0.92271531 -0.77077601 0.64949077], error = 7.322e-02
Iteración 118: x = [ 0.84949077 -0.80126965 0.69189669], error = 7.322e-02
Iteración 119: x = [ 0.89189669 -0.82728045 0.74987441], error = 5.798e-02
Iteración 120: x = [ 0.94987441 -0.79158306 0.69446309], error = 5.798e-02

```

Iteración 121: x = [0.89446309 -0.77644702 0.65433407], error = 5.541e-02
Iteración 122: x = [0.85433407 -0.81418494 0.7173134], error = 6.298e-02
Iteración 123: x = [0.9173134 -0.81850462 0.73857347], error = 6.298e-02
Iteración 124: x = [0.93857347 -0.78169994 0.6734343], error = 6.514e-02
Iteración 125: x = [0.8734343 -0.78735469 0.67057657], error = 6.514e-02
Iteración 126: x = [0.87057657 -0.82063871 0.73288836], error = 6.231e-02
Iteración 127: x = [0.93288836 -0.80648963 0.71910408], error = 6.231e-02
Iteración 128: x = [0.91910408 -0.7787798 0.66386683], error = 5.524e-02
Iteración 129: x = [0.86386683 -0.79948125 0.69150602], error = 5.524e-02
Iteración 130: x = [0.89150602 -0.82019008 0.73639254], error = 4.489e-02
Iteración 131: x = [0.93639254 -0.79514885 0.69839894], error = 4.489e-02
Iteración 132: x = [0.89839894 -0.78220399 0.66603303], error = 3.799e-02
Iteración 133: x = [0.86603303 -0.80929227 0.71049906], error = 4.447e-02
Iteración 134: x = [0.91049906 -0.81435872 0.72932083], error = 4.447e-02
Iteración 135: x = [0.92932083 -0.78742026 0.68232158], error = 4.700e-02
Iteración 136: x = [0.88232158 -0.78975919 0.67696904], error = 4.700e-02
Iteración 137: x = [0.87696904 -0.81459695 0.72279883], error = 4.583e-02
Iteración 138: x = [0.92279883 -0.80581577 0.71573249], error = 4.583e-02
Iteración 139: x = [0.91573249 -0.78466746 0.67429328], error = 4.144e-02
Iteración 140: x = [0.87429328 -0.79856044 0.69193378], error = 4.144e-02
Iteración 141: x = [0.89193378 -0.81486991 0.7264265], error = 3.449e-02
Iteración 142: x = [0.9264265 -0.79742648 0.70063126], error = 3.449e-02
Iteración 143: x = [0.90063126 -0.78662893 0.67486026], error = 2.580e-02
Iteración 144: x = [0.87486026 -0.8059693 0.70605427], error = 3.119e-02
Iteración 145: x = [0.90605427 -0.8110563 0.72215509], error = 3.119e-02
Iteración 146: x = [0.92215509 -0.79143409 0.68841758], error = 3.374e-02
Iteración 147: x = [0.88841758 -0.79181806 0.68212786], error = 3.374e-02
Iteración 148: x = [0.88212786 -0.81025925 0.71567339], error = 3.355e-02
Iteración 149: x = [0.91567339 -0.80501772 0.71274251], error = 3.355e-02
Iteración 150: x = [0.91274251 -0.78897768 0.68181775], error = 3.092e-02
Iteración 151: x = [0.88181775 -0.79817431 0.69276865], error = 3.092e-02
Iteración 152: x = [0.89276865 -0.81089896 0.7190951], error = 2.633e-02
Iteración 153: x = [0.9190951 -0.7988419 0.70178187], error = 2.633e-02
Iteración 154: x = [0.90178187 -0.79000698 0.68148395], error = 2.030e-02
Iteración 155: x = [0.88148395 -0.80373808 0.70321464], error = 2.173e-02
Iteración 156: x = [0.90321464 -0.80845436 0.71664701], error = 2.173e-02
Iteración 157: x = [0.91664701 -0.79423093 0.69255818], error = 2.409e-02
Iteración 158: x = [0.89255818 -0.79353695 0.68623753], error = 2.409e-02
Iteración 159: x = [0.88623753 -0.80716153 0.71067335], error = 2.444e-02
Iteración 160: x = [0.91067335 -0.8042129 0.71018171], error = 2.444e-02
Iteración 161: x = [0.91018171 -0.7921179 0.6872202], error = 2.296e-02
Iteración 162: x = [0.8872202 -0.79810409 0.69375934], error = 2.296e-02
Iteración 163: x = [0.89375934 -0.80795006 0.71372775], error = 1.997e-02
Iteración 164: x = [0.91372775 -0.79968839 0.70226563], error = 1.997e-02
Iteración 165: x = [0.90226563 -0.79256972 0.68642805], error = 1.584e-02
Iteración 166: x = [0.88642805 -0.80226017 0.70144951], error = 1.584e-02
Iteración 167: x = [0.90144951 -0.80642359 0.71244186], error = 1.502e-02
Iteración 168: x = [0.91244186 -0.79616478 0.69533869], error = 1.710e-02
Iteración 169: x = [0.89533869 -0.7949444 0.68947575], error = 1.710e-02
Iteración 170: x = [0.88947575 -0.80496172 0.70718911], error = 1.771e-02
Iteración 171: x = [0.90718911 -0.80346485 0.70804339], error = 1.771e-02
Iteración 172: x = [0.90804339 -0.7943946 0.69107847], error = 1.696e-02
Iteración 173: x = [0.89107847 -0.79820869 0.69475931], error = 1.696e-02
Iteración 174: x = [0.89475931 -0.80577094 0.70981719], error = 1.506e-02
Iteración 175: x = [0.90981719 -0.80016605 0.70235522], error = 1.506e-02
Iteración 176: x = [0.90235522 -0.7945026 0.69009979], error = 1.226e-02
Iteración 177: x = [0.89009979 -0.80129744 0.70039348], error = 1.226e-02
Iteración 178: x = [0.90039348 -0.80485174 0.70925149], error = 1.029e-02
Iteración 179: x = [0.90925149 -0.79749039 0.69718065], error = 1.207e-02
Iteración 180: x = [0.89718065 -0.79607909 0.69200332], error = 1.207e-02

Iteración 181: $x = [0.89200332 \ -0.80340885 \ 0.7047798]$, error = 1.278e-02
Iteración 182: $x = [0.9047798 \ -0.80280339 \ 0.70629226]$, error = 1.278e-02
Iteración 183: $x = [0.90629226 \ -0.79603703 \ 0.6938185]$, error = 1.247e-02
Iteración 184: $x = [0.8938185 \ -0.79839924 \ 0.69568922]$, error = 1.247e-02
Iteración 185: $x = [0.89568922 \ -0.80416844 \ 0.70698188]$, error = 1.129e-02
Iteración 186: $x = [0.90698188 \ -0.80040992 \ 0.70222656]$, error = 1.129e-02
Iteración 187: $x = [0.90222656 \ -0.79595242 \ 0.69281316]$, error = 9.413e-03
Iteración 188: $x = [0.89281316 \ -0.80068343 \ 0.69979723]$, error = 9.413e-03
Iteración 189: $x = [0.89979723 \ -0.80364411 \ 0.70684512]$, error = 7.048e-03
Iteración 190: $x = [0.90684512 \ -0.7983901 \ 0.69838071]$, error = 8.464e-03
Iteración 191: $x = [0.89838071 \ -0.79698226 \ 0.69395983]$, error = 8.464e-03
Iteración 192: $x = [0.89395983 \ -0.80231969 \ 0.70312816]$, error = 9.168e-03
Iteración 193: $x = [0.90312816 \ -0.80223805 \ 0.70488033]$, error = 9.168e-03
Iteración 194: $x = [0.90488033 \ -0.79721584 \ 0.69575282]$, error = 9.128e-03
Iteración 195: $x = [0.89575282 \ -0.79862163 \ 0.69651175]$, error = 9.128e-03
Iteración 196: $x = [0.89651175 \ -0.80299565 \ 0.70493637]$, error = 8.425e-03
Iteración 197: $x = [0.90493637 \ -0.80051003 \ 0.70199042]$, error = 8.425e-03
Iteración 198: $x = [0.90199042 \ -0.79703421 \ 0.69480862]$, error = 7.182e-03
Iteración 199: $x = [0.89480862 \ -0.80030263 \ 0.69949247]$, error = 7.182e-03
Iteración 200: $x = [0.89949247 \ -0.80272257 \ 0.70504007]$, error = 5.548e-03
Iteración 201: $x = [0.90504007 \ -0.79899375 \ 0.69914624]$, error = 5.894e-03
Iteración 202: $x = [0.89914624 \ -0.79769341 \ 0.69546306]$, error = 5.894e-03
Iteración 203: $x = [0.89546306 \ -0.80156111 \ 0.70200705]$, error = 6.544e-03
Iteración 204: $x = [0.90200705 \ -0.80176671 \ 0.70375638]$, error = 6.544e-03
Iteración 205: $x = [0.90375638 \ -0.79805738 \ 0.69710959]$, error = 6.647e-03
Iteración 206: $x = [0.89710959 \ -0.79884441 \ 0.69721493]$, error = 6.647e-03
Iteración 207: $x = [0.89721493 \ -0.80214147 \ 0.7034682]$, error = 6.253e-03
Iteración 208: $x = [0.9034682 \ -0.80052548 \ 0.70171433]$, error = 6.253e-03
Iteración 209: $x = [0.90171433 \ -0.79783732 \ 0.69626906]$, error = 5.445e-03
Iteración 210: $x = [0.89626906 \ -0.80007557 \ 0.69936701]$, error = 5.445e-03
Iteración 211: $x = [0.89936701 \ -0.80202372 \ 0.70369316]$, error = 4.326e-03
Iteración 212: $x = [0.90369316 \ -0.79939321 \ 0.69962113]$, error = 4.326e-03
Iteración 213: $x = [0.89962113 \ -0.79824814 \ 0.69661024]$, error = 4.072e-03
Iteración 214: $x = [0.89661024 \ -0.80103687 \ 0.7012548]$, error = 4.645e-03
Iteración 215: $x = [0.9012548 \ -0.80138118 \ 0.70287133]$, error = 4.645e-03
Iteración 216: $x = [0.90287133 \ -0.79865477 \ 0.69805461]$, error = 4.817e-03
Iteración 217: $x = [0.89805461 \ -0.79905068 \ 0.69780129]$, error = 4.817e-03
Iteración 218: $x = [0.89780129 \ -0.80152237 \ 0.70242005]$, error = 4.619e-03
Iteración 219: $x = [0.90242005 \ -0.80049434 \ 0.70143752]$, error = 4.619e-03
Iteración 220: $x = [0.90143752 \ -0.79843059 \ 0.69733278]$, error = 4.105e-03
Iteración 221: $x = [0.89733278 \ -0.79994804 \ 0.69934718]$, error = 4.105e-03
Iteración 222: $x = [0.89934718 \ -0.80149682 \ 0.7026932]$, error = 3.346e-03
Iteración 223: $x = [0.9026932 \ -0.79965311 \ 0.69990441]$, error = 3.346e-03
Iteración 224: $x = [0.89990441 \ -0.7986773 \ 0.69748025]$, error = 2.789e-03
Iteración 225: $x = [0.89748025 \ -0.80067773 \ 0.70075694]$, error = 3.277e-03
Iteración 226: $x = [0.90075694 \ -0.80107064 \ 0.70218089]$, error = 3.277e-03
Iteración 227: $x = [0.90218089 \ -0.79907631 \ 0.69870774]$, error = 3.473e-03
Iteración 228: $x = [0.89870774 \ -0.79923262 \ 0.69828096]$, error = 3.473e-03
Iteración 229: $x = [0.89828096 \ -0.80107589 \ 0.70167595]$, error = 3.395e-03
Iteración 230: $x = [0.90167595 \ -0.80044053 \ 0.7011811]$, error = 3.395e-03
Iteración 231: $x = [0.9011811 \ -0.79886675 \ 0.69810378]$, error = 3.077e-03
Iteración 232: $x = [0.89810378 \ -0.7998835 \ 0.69938553]$, error = 3.077e-03
Iteración 233: $x = [0.89938553 \ -0.80110173 \ 0.70195446]$, error = 2.569e-03
Iteración 234: $x = [0.90195446 \ -0.79981862 \ 0.70006361]$, error = 2.569e-03
Iteración 235: $x = [0.90006361 \ -0.79900687 \ 0.69813623]$, error = 1.927e-03
Iteración 236: $x = [0.89813623 \ -0.80043414 \ 0.70043296]$, error = 2.297e-03
Iteración 237: $x = [0.90043296 \ -0.80082365 \ 0.7016467]$, error = 2.297e-03
Iteración 238: $x = [0.9016467 \ -0.79937185 \ 0.69915522]$, error = 2.491e-03
Iteración 239: $x = [0.89915522 \ -0.79938784 \ 0.69866737]$, error = 2.491e-03
Iteración 240: $x = [0.89866737 \ -0.80075555 \ 0.70115086]$, error = 2.483e-03

Iteración 241: $x = [0.90115086 \ -0.8003786 \ 0.70095485]$, error = 2.483e-03
Iteración 242: $x = [0.90095485 \ -0.79918586 \ 0.69865984]$, error = 2.295e-03
Iteración 243: $x = [0.89865984 \ -0.79985761 \ 0.69945222]$, error = 2.295e-03
Iteración 244: $x = [0.89945222 \ -0.80080702 \ 0.70141135]$, error = 1.959e-03
Iteración 245: $x = [0.90141135 \ -0.79992105 \ 0.70014427]$, error = 1.959e-03
Iteración 246: $x = [0.90014427 \ -0.79925826 \ 0.69862812]$, error = 1.516e-03
Iteración 247: $x = [0.89862812 \ -0.80027083 \ 0.7002266]$, error = 1.598e-03
Iteración 248: $x = [0.9002266 \ -0.80062929 \ 0.70123646]$, error = 1.598e-03
Iteración 249: $x = [0.90123646 \ -0.79957758 \ 0.69945875]$, error = 1.778e-03
Iteración 250: $x = [0.89945875 \ -0.79951708 \ 0.69897475]$, error = 1.778e-03
Iteración 251: $x = [0.89897475 \ -0.80052694 \ 0.70078271]$, error = 1.808e-03
Iteración 252: $x = [0.90078271 \ -0.80031695 \ 0.70076178]$, error = 1.808e-03
Iteración 253: $x = [0.90076178 \ -0.7994182 \ 0.69905882]$, error = 1.703e-03
Iteración 254: $x = [0.89905882 \ -0.7998544 \ 0.69952911]$, error = 1.703e-03
Iteración 255: $x = [0.89952911 \ -0.80058831 \ 0.70101398]$, error = 1.485e-03
Iteración 256: $x = [0.90101398 \ -0.79998195 \ 0.70017673]$, error = 1.485e-03
Iteración 257: $x = [0.90017673 \ -0.79944883 \ 0.69899505]$, error = 1.182e-03
Iteración 258: $x = [0.89899505 \ -0.80016287 \ 0.70009886]$, error = 1.182e-03
Iteración 259: $x = [0.90009886 \ -0.80047776 \ 0.70092352]$, error = 1.104e-03
Iteración 260: $x = [0.90092352 \ -0.79971969 \ 0.69966226]$, error = 1.261e-03
Iteración 261: $x = [0.89966226 \ -0.79962268 \ 0.69921664]$, error = 1.261e-03
Iteración 262: $x = [0.89921664 \ -0.80036471 \ 0.7005264]$, error = 1.310e-03
Iteración 263: $x = [0.9005264 \ -0.80026008 \ 0.70060101]$, error = 1.310e-03
Iteración 264: $x = [0.90060101 \ -0.79958655 \ 0.69934356]$, error = 1.257e-03
Iteración 265: $x = [0.89934356 \ -0.79986361 \ 0.69960572]$, error = 1.257e-03
Iteración 266: $x = [0.89960572 \ -0.80042679 \ 0.70072464]$, error = 1.119e-03
Iteración 267: $x = [0.90072464 \ -0.80001598 \ 0.70018089]$, error = 1.119e-03
Iteración 268: $x = [0.90018089 \ -0.79959246 \ 0.69926737]$, error = 9.135e-04
Iteración 269: $x = [0.89926737 \ -0.80009271 \ 0.70002288]$, error = 9.135e-04
Iteración 270: $x = [0.90002288 \ -0.80036059 \ 0.70068627]$, error = 7.555e-04
Iteración 271: $x = [0.90068627 \ -0.79981699 \ 0.69979682]$, error = 8.895e-04
Iteración 272: $x = [0.89979682 \ -0.79970766 \ 0.69940523]$, error = 8.895e-04
Iteración 273: $x = [0.89940523 \ -0.80025028 \ 0.70034935]$, error = 9.441e-04
Iteración 274: $x = [0.90034935 \ -0.80021005 \ 0.70046963]$, error = 9.441e-04
Iteración 275: $x = [0.90046963 \ -0.79970792 \ 0.69954563]$, error = 9.240e-04
Iteración 276: $x = [0.89954563 \ -0.79987878 \ 0.69967642]$, error = 9.240e-04
Iteración 277: $x = [0.89967642 \ -0.80030808 \ 0.70051498]$, error = 8.386e-04
Iteración 278: $x = [0.90051498 \ -0.80003305 \ 0.70016954]$, error = 8.386e-04
Iteración 279: $x = [0.90016954 \ -0.79970012 \ 0.69946849]$, error = 7.010e-04
Iteración 280: $x = [0.89946849 \ -0.8000481 \ 0.6999804]$, error = 7.010e-04
Iteración 281: $x = [0.8999804 \ -0.80027065 \ 0.70050745]$, error = 5.271e-04
Iteración 282: $x = [0.90050745 \ -0.79988294 \ 0.69988428]$, error = 6.232e-04
Iteración 283: $x = [0.89988428 \ -0.7997752 \ 0.69955108]$, error = 6.232e-04
Iteración 284: $x = [0.89955108 \ -0.80017009 \ 0.70022812]$, error = 6.770e-04
Iteración 285: $x = [0.90022812 \ -0.80016743 \ 0.70036388]$, error = 6.770e-04
Iteración 286: $x = [0.90036388 \ -0.79979497 \ 0.69968816]$, error = 6.757e-04
Iteración 287: $x = [0.89968816 \ -0.79989602 \ 0.69973864]$, error = 6.757e-04
Iteración 288: $x = [0.89973864 \ -0.80022126 \ 0.70036383]$, error = 6.252e-04
Iteración 289: $x = [0.90036383 \ -0.80003972 \ 0.70015073]$, error = 6.252e-04
Iteración 290: $x = [0.90015073 \ -0.7997804 \ 0.69961631]$, error = 5.344e-04
Iteración 291: $x = [0.89961631 \ -0.80002056 \ 0.69995907]$, error = 5.344e-04
Iteración 292: $x = [0.89995907 \ -0.80020208 \ 0.70037341]$, error = 4.143e-04
Iteración 293: $x = [0.90037341 \ -0.79992711 \ 0.6999399]$, error = 4.335e-04
Iteración 294: $x = [0.8999399 \ -0.79982832 \ 0.69966303]$, error = 4.335e-04
Iteración 295: $x = [0.89966303 \ -0.80011429 \ 0.70014594]$, error = 4.829e-04
Iteración 296: $x = [0.90014594 \ -0.800132 \ 0.70027982]$, error = 4.829e-04
Iteración 297: $x = [0.90027982 \ -0.79985707 \ 0.69978806]$, error = 4.918e-04
Iteración 298: $x = [0.89978806 \ -0.79991308 \ 0.69979164]$, error = 4.918e-04
Iteración 299: $x = [0.89979164 \ -0.80015806 \ 0.70025541]$, error = 4.638e-04
Iteración 300: $x = [0.90025541 \ -0.80004033 \ 0.70012933]$, error = 4.638e-04

⚠ No se alcanzó la tolerancia después de 300 iteraciones.

Resultado aproximado: [0.90025541 -0.80004033 0.70012933]

Ejercicio 8

Un cable coaxial está formado por un conductor interno de 0.1 pulgadas cuadradas y un conductor externo de 0.5 pulgadas cuadradas. El potencial en un punto en la sección transversal del cable se describe mediante la ecuación de Laplace. Suponga que el conductor interno se mantiene en 0 volts y el conductor externo se mantiene en 110 volts. Aproximar el potencial entre los dos conductores requiere resolver el siguiente sistema lineal.

$$\begin{bmatrix} 4 & -1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 4 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 4 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 4 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 4 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 4 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & 4 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 4 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & -1 & 4 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \\ w_5 \\ w_6 \\ w_7 \\ w_8 \\ w_9 \\ w_{10} \\ w_{11} \\ w_{12} \end{bmatrix} = \begin{bmatrix} 220 \\ 110 \\ 110 \\ 220 \\ 110 \\ 110 \\ 110 \\ 110 \\ 220 \\ 110 \\ 110 \\ 220 \end{bmatrix}.$$

a. ¿La matriz es estrictamente diagonalmente dominante?

```
In [28]: import numpy as np

# Matriz de coeficientes A de 12x12
A = np.zeros((12, 12))

# Llenamos la diagonal principal con 4
np.fill_diagonal(A, 4)

# Llenamos las super y sub diagonales con -1
for i in range(11):
    A[i, i+1] = -1
    A[i+1, i] = -1

# Agregamos los elementos no estándar de la matriz
A[0, 4] = -1
A[4, 0] = -1
A[9, 11] = -1
A[11, 9] = -1

# Verificamos la dominancia diagonal para cada fila
print("--- Verificación de la dominancia diagonal para cada fila ---")
for i in range(A.shape[0]):
    diagonal_element = abs(A[i, i])
    off_diagonal_sum = np.sum(abs(A[i, :])) - diagonal_element
```

```

comparacion = diagonal_element > off_diagonal_sum

print(f"Fila {i+1}: |{diagonal_element}| > ({off_diagonal_sum}) -> {comparacion}")

--- Verificación de la dominancia diagonal para cada fila ---
Fila 1: |4.0| > (2.0) -> True
Fila 2: |4.0| > (2.0) -> True
Fila 3: |4.0| > (2.0) -> True
Fila 4: |4.0| > (2.0) -> True
Fila 5: |4.0| > (3.0) -> True
Fila 6: |4.0| > (2.0) -> True
Fila 7: |4.0| > (2.0) -> True
Fila 8: |4.0| > (2.0) -> True
Fila 9: |4.0| > (2.0) -> True
Fila 10: |4.0| > (3.0) -> True
Fila 11: |4.0| > (2.0) -> True
Fila 12: |4.0| > (2.0) -> True

```

Si la matriz tiene una diagonal dominante.

b. Resuelva el sistema lineal usando el método de Jacobi con $x(0) = 0$ y $TOL = 10^{-2}$.

```

In [29]: import numpy as np

def jacobi_solver(A, b, initial_guess, tolerance=1e-2, max_iterations=200):
    """
    Resuelve un sistema de ecuaciones lineales Ax = b usando el método de Jacobi.

    Args:
        A (np.array): La matriz de coeficientes.
        b (np.array): El vector de constantes.
        initial_guess (np.array): El vector de valores iniciales para la solución.
        tolerance (float): El umbral de convergencia.
        max_iterations (int): El número máximo de iteraciones.

    Returns:
        np.array: El vector de solución aproximada.
        tuple: (mensaje de error, None) si no converge.
    """
    n = len(b)
    x = initial_guess.copy()

    for k in range(max_iterations):
        x_new = np.zeros(n)
        for i in range(n):
            sum_val = 0
            for j in range(n):
                if i != j:
                    sum_val += A[i, j] * x[j]
            x_new[i] = (b[i] - sum_val) / A[i, i]

        if np.linalg.norm(x_new - x) < tolerance:
            print(f"Convergencia alcanzada en {k+1} iteraciones.")
            return x_new

        x = x_new

    return "No se alcanzó la convergencia dentro del número máximo de iteraciones."

# --- Definición y Solución del Sistema ---

```

```

# Definimos la matriz A de 12x12 explícitamente para mayor claridad
n = 12
A = np.zeros((n, n))

# Llenamos la diagonal principal
np.fill_diagonal(A, 4)

# Llenamos las super y sub diagonales
for i in range(n - 1):
    A[i, i+1] = -1
    A[i+1, i] = -1

# Corregimos y agregamos los elementos no estándar de la matriz
A[0, 4] = -1
A[4, 0] = -1
A[9, 11] = -1
A[11, 9] = -1

# El vector de constantes b
b = np.array([220, 110, 110, 110, 220, 110, 110, 110, 220, 110, 110, 220])

# Parámetros para el método de Jacobi
initial_guess = np.zeros(n)
tolerance = 1e-2

# Resolvemos el sistema
print("--- Solución del sistema de 12x12 por el método de Jacobi ---")
solucion = jacobi_solver(A, b, initial_guess, tolerance)

if isinstance(solucion, np.ndarray):
    print("El sistema tiene una solución aproximada:")
    for i in range(n):
        print(f"w{i+1:<2} = {solucion[i]:.4f}")
else:
    print(solucion)

```

```

--- Solución del sistema de 12x12 por el método de Jacobi ---
Convergencia alcanzada en 18 iteraciones.
El sistema tiene una solución aproximada:
w1  = 101.2224
w2  = 68.4768
w3  = 62.6873
w4  = 72.2750
w5  = 116.4159
w6  = 72.1706
w7  = 62.2698
w8  = 66.9115
w9  = 95.3795
w10 = 94.6108
w11 = 75.5352
w12 = 97.5352

```

c. Repita la parte b) mediante el método de Gauss-Siedel.

```

In [30]: import numpy as np

def gauss_seidel_solver(A, b, initial_guess, tolerance=1e-2, max_iterations=200)
    """
    Resuelve un sistema de ecuaciones lineales Ax = b usando el método de Gauss-

```



```

Args:
    A (np.array): La matriz de coeficientes.
    b (np.array): El vector de constantes.
    initial_guess (np.array): El vector de valores iniciales para la solución.
    tolerance (float): El umbral de convergencia.
    max_iterations (int): El número máximo de iteraciones.

Returns:
    np.array: El vector de solución aproximada.
    tuple: (mensaje de error, None) si no converge.
"""
n = len(b)
x = initial_guess.copy()

for k in range(max_iterations):
    x_new = x.copy()
    for i in range(n):
        sum_val = 0
        for j in range(n):
            if i != j:
                sum_val += A[i, j] * x_new[j]
        x_new[i] = (b[i] - sum_val) / A[i, i]

    if np.linalg.norm(x_new - x) < tolerance:
        print(f"Convergencia alcanzada en {k+1} iteraciones.")
        return x_new

    x = x_new

return "No se alcanzó la convergencia dentro del número máximo de iteraciones"

# --- Definición y Solución del Sistema ---

# Definimos la matriz A de 12x12 explícitamente para mayor claridad
n = 12
A = np.zeros((n, n))

# Llenamos la diagonal principal
np.fill_diagonal(A, 4)

# Llenamos las super y sub diagonales
for i in range(n - 1):
    A[i, i+1] = -1
    A[i+1, i] = -1

# Agregamos los elementos no estándar de la matriz
A[0, 4] = -1
A[4, 0] = -1
A[9, 11] = -1
A[11, 9] = -1

# El vector de constantes b
b = np.array([220, 110, 110, 110, 220, 110, 110, 110, 220, 110, 110, 220])

# Parámetros para el método de Gauss-Seidel
initial_guess = np.zeros(n)
tolerance = 1e-2

# Resolvemos el sistema

```

```
print("--- Solución del sistema de 12x12 por el método de Gauss-Seidel ---")
solucion = gauss_seidel_solver(A, b, initial_guess, tolerance)

if isinstance(solucion, np.ndarray):
    print("El sistema tiene una solución aproximada:")
    for i in range(n):
        print(f"w{i+1:<2} = {solucion[i]:.4f}")
else:
    print(solucion)
```

```
--- Solución del sistema de 12x12 por el método de Gauss-Seidel ---
Convergencia alcanzada en 11 iteraciones.
El sistema tiene una solución aproximada:
w1  = 101.2217
w2  = 68.4765
w3  = 62.6871
w4  = 72.2750
w5  = 116.4165
w6  = 72.1713
w7  = 62.2706
w8  = 66.9126
w9  = 95.3813
w10 = 94.6140
w11 = 75.5378
w12 = 97.5379
```